ANSI SQL Using MySQL Exercises

1. User Upcoming Events Show a list of all upcoming events a user is registered for in their city, sorted by date.

```
SELECT e.*
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
JOIN Users u ON u.user_id = r.user_id
WHERE e.status = 'upcoming' AND e.city = u.city
ORDER BY e.start_date;
```

2. Top Rated Events Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```
SELECT e.event_id, e.title, AVG(f.rating) avg_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

3. Inactive Users Retrieve users who have not registered for any events in the last 90 days.

```
SELECT *
FROM Users u
WHERE u.user_id NOT IN (
  SELECT r.user_id
  FROM Registrations r
  WHERE r.registration_date >= CURDATE() - INTERVAL 90 DAY
);
```

4. Peak Session Hours Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```
SELECT event_id, COUNT(*) session_count
```

```
FROM Sessions

WHERE HOUR(start_time) BETWEEN 10 AND 11

GROUP BY event_id;
```

5. Most Active Cities List the top 5 cities with the highest number of distinct user registrations.

```
SELECT city, COUNT(DISTINCT user_id) total_users

FROM Users

GROUP BY city

ORDER BY total_users DESC

LIMIT 5;
```

6. Event Resource Summary Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.

```
SELECT event_id,

SUM(resource_type = 'pdf') pdfs,

SUM(resource_type = 'image') images,

SUM(resource_type = 'link') links

FROM Resources

GROUP BY event_id;
```

7. Low Feedback Alerts List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```
SELECT u.full_name, e.title, f.comments

FROM Feedback f

JOIN Users u ON f.user_id = u.user_id

JOIN Events e ON f.event_id = e.event_id

WHERE f.rating < 3;
```

8. Sessions per Upcoming Event Display all upcoming events with the count of sessions scheduled for them.

```
SELECT e.event_id, e.title, COUNT(s.session_id) session_count

FROM Events e
```

```
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;
```

9. Organizer Event Summary For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```
SELECT organizer_id, status, COUNT(*) total_events
FROM Events
GROUP BY organizer_id, status;
```

10. Feedback Gap Identify events that had registrations but received no feedback at all.

```
SELECT e.event_id, e.title
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE f.feedback_id IS NULL
GROUP BY e.event_id, e.title;
```

11. Daily New User Count Find the number of users who registered each day in the last 7 days.

```
SELECT registration_date, COUNT(*) user_count
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date;
```

12. Event with Maximum Sessions List the event(s) with the highest number of sessions.

```
SELECT event_id
FROM Sessions
GROUP BY event_id
HAVING COUNT(*) = (
  SELECT MAX(cnt)
  FROM (
```

```
    SELECT COUNT(*) cnt FROM Sessions GROUP BY event_id
  ) t
);
```

13. Average Rating per City Calculate the average feedback rating of events conducted in each city.

```
SELECT e.city, AVG(f.rating) avg_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city;
```

14. Most Registered Events List top 3 events based on the total number of user registrations.

```
SELECT e.event_id, e.title, COUNT(r.user_id) total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

15. Event Session Time Conflict Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

```
SELECT s1.event_id, s1.session_id, s2.session_id
FROM Sessions s1
JOIN Sessions s2
ON s1.event_id = s2.event_id
AND s1.session_id < s2.session_id
AND s1.start_time < s2.end_time
AND s2.start_time < s1.end_time;
```

16. Unregistered Active Users Find users who created an account in the last 30 days but haven’t registered for any events.

```
SELECT *
FROM Users u
```

```
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY
AND u.user_id NOT IN (SELECT user_id FROM Registrations);
```

17. Multi-Session Speakers Identify speakers who are handling more than one session across all events.

```
SELECT speaker_name, COUNT(*) total_sessions
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(*) > 1;
```

18. Resource Availability Check List all events that do not have any resources uploaded.

```
SELECT e.event_id, e.title
FROM Events e
LEFT JOIN Resources r ON e.event_id = r.event_id
WHERE r.resource_id IS NULL;
```

19. Completed Events with Feedback Summary For completed events, show total registrations and average feedback rating.

```
SELECT e.event_id, e.title,
COUNT(DISTINCT r.registration_id) total_registrations,
AVG(f.rating) avg_rating
FROM Events e
LEFT JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title;
```

20. User Engagement Index For each user, calculate how many events they attended and how many feedbacks they submitted.

```
SELECT u.user_id,
COUNT(DISTINCT r.event_id) events_attended,
COUNT(DISTINCT f.feedback_id) feedbacks_submitted
```

```
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
LEFT JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id;
```

21. Top Feedback Providers List top 5 users who have submitted the most feedback entries.

```
SELECT user_id, COUNT(*) total_feedbacks
FROM Feedback
GROUP BY user_id
ORDER BY total_feedbacks DESC
LIMIT 5;
```

22. Duplicate Registrations Check Detect if a user has been registered more than once for the same event

```
SELECT user_id, event_id, COUNT(*) cnt
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;
```

23. Registration Trends Show a month-wise registration count trend over the past 12 months.

```
SELECT DATE_FORMAT(registration_date, '%Y-%m') month, COUNT(*) total
FROM Registrations
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY month
ORDER BY month;
```

24. Average Session Duration per Event Compute the average duration (in minutes) of sessions in each event.

```
SELECT event_id,
AVG(TIMESTAMPDIFF(MINUTE, start_time, end_time)) avg_duration
FROM Sessions
GROUP BY event_id;
```

25. Events Without Sessions List all events that currently have no sessions scheduled under them.

```
SELECT e.event_id, e.title
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE s.session_id IS NULL;
```